COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 03

Organizing data and small programs

Four expanded lessons with worked examples, practical assignments and five-question quizzes. Use the main workbook for this module’s project and additional exercises.

4 lessons · 4 demonstrations · 20 lesson questions

Package 02 · Expanded teaching edition · September 28, 2026
Volume 01 of the full undergraduate-to-graduate program
Self-study material. Completion requires implementation, testing and review.

Contents

Use the linked contents or PDF bookmarks to navigate.

Module 03: Organizing data and small programs 3
Lesson 03.1: Sequences, dictionaries and identity 3
Lesson 03.2: Decomposition, modules and interfaces 6
Lesson 03.3: Files, serialization and validation 9
Lesson 03.4: Classes, composition and state machines 12

Module 03: Organizing data and small programs

Prerequisites: Module 02.

Integration project: A data-driven simulator with validated persistence. The full brief and review are in the Lab Workbook.

Lesson 03.1: Sequences, dictionaries and identity

Learning outcome: Choose collections according to the operations the program needs.

Organize data around the questions you ask

A list preserves an ordered sequence of elements. A dictionary maps keys to values. Choose according to the operations your game needs, not according to which syntax looks shorter. A turn order may naturally be a list; finding a unit by its stable identifier may naturally be a dictionary. One game can use both representations for different purposes.

A list index identifies a position in the current ordering, not necessarily a persistent game entity. Removing or sorting entries can change which unit occupies index 2. A unit name is also a poor persistent identity if the player can rename the unit or two units can share a name. Give each unit a stable ID and treat the displayed name as ordinary editable data.

Distinguish rebinding from mutation

With `other = units`, both names refer to the same list. Appending through one name changes the object observed through the other. This is different from rebinding an integer name in Lesson 01.3. With `other = list(units)`, the outer list is new, but its elements still refer to the same objects. If those elements are dictionaries, mutating one dictionary is visible from both lists.

For the flat records in this lesson, copying each record with `dict(unit)` is sufficient to make independent copies of its ID, name and health fields, because the field values are immutable integers and strings. It would not recursively copy a nested mutable inventory list. That limitation is part of the contract; do not call a shallow copy a universal isolation mechanism.

Filter without corrupting the source

To select living units, build a new result containing units whose health is greater than zero. Avoid deleting entries from the same list you are currently walking through: the remaining indices can shift and cause an element to be skipped. A separate result is easier to reason about and preserves the original collection for comparison.

There are two legitimate filter designs: a new list of references to the original records, or a new list of copied records. This package's lab chooses copied flat records so that renaming a filtered record does not edit the source. Another application might deliberately choose shared identity. State the choice explicitly rather than discovering it accidentally through a bug.

Reject ambiguous identity

When constructing the ID index, reject duplicates. Silently replacing an earlier record with the later one may hide a data-entry or loading problem. The lookup can only be meaningful if the ID identifies one unit under the chosen schema. An empty collection is valid and should produce an empty index and an empty living-unit list.

An index contains a view of data at a point in time. If it contains copies, later edits to the source records will not automatically update those copies. A production design must decide who owns the authoritative record and whether other structures hold references, copies or IDs. This small lab teaches the tradeoff without pretending there is one representation suitable for every engine.

Test relationships as well as values

A test that only compares resulting text can miss aliasing. Record the source, create the filtered result, edit a result record and check whether the source changed. Likewise, verify that changing a name does not change the ID used for lookup. Check two living units with the same name but distinct IDs.

Keep the data set small enough to inspect manually: five units with a mixture of positive and zero health. The important work is not the volume of data; it is the clarity of identity, ownership and mutation rules. Those same questions later determine whether entity handles and resource references are reliable.

Worked example

Predict before running `examples/example_03_1.py`.

```
units = [
    {"id": 101, "name": "Scout", "health": 8},
    {"id": 102, "name": "Scout", "health": 0},
]
alias = units
copies = [dict(unit) for unit in units]
alias[0]["name"] = "Runner"
copies[0]["name"] = "Independent"
print("original name:", units[0]["name"])
print("copied name:", copies[0]["name"])
print("living IDs:", [unit["id"] for unit in units if unit["health"] > 0])
```

The alias shares the original list, so changing its first dictionary changes the source. The copied records are separate dictionaries, so their string-valued name edits remain separate. Two records start with the same name, but their IDs remain distinct. This example does not contain nested mutable fields; extending it to those fields requires another ownership decision.

Expected demonstration output

```
original name: Runner
copied name: Independent
living IDs: [101]
```

Practical work and pass evidence

Create five flat unit records and implement `index_units` and `living_units` in `labs/student/lab_03_1.py`. Use stable IDs, reject duplicate IDs and return independent shallow copies of flat records. Preserve the input collection while filtering.

Test an empty collection, all-dead units, duplicate names with different IDs and duplicate IDs. Rename a copied living unit and demonstrate that the original record is unchanged. Explain why the same technique would not isolate a nested inventory list. Keep the small input data and the before/after snapshots in your journal.

Changed-case challenge

Add a nested list of item IDs in a separate experiment and demonstrate the limit of a shallow record copy. Document one design that would make ownership clear.

Companion source study

[S07] Python: Data Structures; [S09] Python: Classes. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. When is a dictionary preferable to a list?
2. Does b = a copy a Python list?
3. Why can a shallow copy still share changes?
4. After `b = list(a)`, where a contains dictionaries, does changing `b[0]["health"]` necessarily leave a unchanged?
5. Two units share the name Scout but have IDs 101 and 102. Must an ID index reject them?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.2: Decomposition, modules and interfaces

Learning outcome: Divide a small game into parts that can be tested independently.

Separate policy from interaction

A resource rule answers a question such as “what balance remains after this spending request?” A terminal interface answers different questions: how to ask for input, how to display errors and when to stop. Combining all of these in one loop makes the arithmetic difficult to test without simulated typing. Separating them gives each part a narrower responsibility.

Our rule function receives two nonnegative integers. When the request is affordable it returns the remainder; otherwise it returns the unchanged balance. The interface may print “not enough resources,” but the rule function does not need to know what a terminal is. This is a small example of a boundary that later separates game simulation from rendering and platform services.

A module is an importable unit of code

A Python source file can be imported as a module. Other code then accesses its functions and constants through the module namespace or explicit imported names. Importing executes top-level statements, so a top-level call to `input()` can unexpectedly start a game during a test. Keep interaction behind a callable entry point and the `if __name__ == "__main__":` guard when a file serves both as a script and an importable module.

The guard asks whether the file is being run as the main entry point rather than imported in the usual way. It does not mean the whole module is inert: function definitions, imports and other top-level initialization still execute. Keep that initialization small, predictable and free of prompts or unrelated writes.

Specify the direction of dependencies

Let the interface import the simulation, not the simulation import the interface. Tests should be able to import the simulation without pulling in terminal prompts. If two modules import one another because both want to control the game loop, the design may lack a clear owner for orchestration. Move that coordination to an entry point that calls the lower-level operations.

A tiny application does not need a large framework to establish this direction. Two files can be enough: `simulation.py` with the rule and `main.py` with interaction. Add another module when it has a coherent responsibility, not just to make the project look professional.

Refactor behavior, not just text

Refactoring changes structure while preserving a defined behavior. Before moving code, save representative inputs and expected results. After the change, rerun those cases against the extracted function. When a refactor also changes a rule, separate and name that change so a reviewer can reason about it.

The first checkpoint is that importing the simulation neither prints nor prompts. The second is that the same input sequence produces the same state sequence. The third is that the command-line entry point still handles invalid input usefully. A clean folder structure alone proves none of these things.

Define failures at the appropriate layer

The rule rejects negative or non-integer values with `ValueError`; the parser handles text conversion. Under the strict lab schema, Boolean values are rejected as counts even though they participate in Python's integer hierarchy. Unaffordable but otherwise valid requests are not parser errors; they are valid game requests whose defined outcome is no change.

This distinction prevents a common design mistake: using exceptions for every ordinary game outcome while silently accepting truly invalid state. Decide what is an expected branch in the game and what violates an interface contract. Tests should make that distinction visible.

Worked example

Predict before running `examples/example_03_2.py`.

```
def spend(resources, cost):
    if resources < 0 or cost < 0:
        raise ValueError("nonnegative amounts required")
    if cost > resources:
        return resources
    return resources - cost

def main():
    resources = 8
    for cost in (5, 5, 0):
        resources = spend(resources, cost)
        print("cost:", cost, "remaining:", resources)

if __name__ == "__main__":
    main()
```

This miniature example uses trusted integer inputs; the student lab additionally validates the strict types. The three requests demonstrate affordable spending, a rejected purchase and a zero-cost request. Importing this example defines functions but does not call `main`. Running it directly calls the entry point once.

Expected demonstration output

```
cost: 5 remaining: 3
cost: 5 remaining: 3
cost: 0 remaining: 3
```

Practical work and pass evidence

Implement the strict `spend` contract in `labs/student/lab_03_2.py`. Refactor your earlier resource simulator into a rule module and a separate command-line entry point in `workspace/module03_simulator/`. The rule module must be importable without output or prompts.

Save a before-refactor trace and compare it with the after-refactor trace. Add direct function tests for exact affordability, insufficient balance, zero cost and invalid input. Pass by drawing the dependency direction and showing an independent test that uses the rule without launching the interface.

Changed-case challenge

Create a second caller that feeds a fixed list of requests without user input. It should reuse the same rule module rather than copy its arithmetic.

Companion source study

[S10] Python: Modules; [S08] Python: Errors and Exceptions. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What is an interface?
2. Why avoid starting a game as a side effect of importing simulation.py?
3. How does command-as-data design help replays?
4. Why can placing `input()` at module top level make automated tests hang?
5. A refactor changes a rejected purchase from no change to a negative balance. Is that merely an internal restructuring?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.3: Files, serialization and validation

Learning outcome: Save structured data without assuming every file is trustworthy.

Serialization is a boundary crossing

Serialization converts structured information into a representation suitable for storage or transmission. JSON is a text format with numbers, strings, Booleans, null, arrays and objects. Parsing JSON establishes that the text has a recognizable structure; it does not establish that it describes a valid game state.

Our lesson save has exactly three fields: version, x and health. Version is integer 1; x is an integer from -1000 through 1000; health is an integer from 0 through 100. Additional fields are rejected in this small schema. A different format could allow extensions, but it must say how unknown fields are handled. The exact-key policy here makes mismatched versions easier to expose.

Validate shape, types, ranges and relationships

Validation proceeds in layers. First, the root must be an object/dictionary. Second, the required keys must be present and unexpected keys absent. Third, values must have the required types. Fourth, ranges and version must be supported. A numeric-looking string is not an integer under this schema. A Boolean is also not an acceptable health count or version, so the reference uses an exact integer type check rather than accepting the whole bool/int relationship.

Some states require relational checks too. The capstone later requires remaining energy plus spent turns to equal the starting energy. Checking each field individually would not detect every inconsistent pair. State validity is more than a list of independent minimums and maximums.

Build a candidate before replacing live state

Parse and validate into a new candidate. Replace the live state only after the complete operation succeeds. In `live = decode_save(text)`, assignment happens after the function returns. If decoding raises a handled error, the old binding can remain available. Do not clear the live state first and hope decoding succeeds.

Loading should not rewrite the save file. A truncated or malformed input must not destroy the known-good save you were trying to read. Test the failed-load path explicitly: save a valid state, attempt to load invalid data and verify that both the existing working state and the good saved bytes remain unchanged where the test applies.

Parsing limits and duplicate keys

Even text with valid JSON syntax can be unreasonable to process. This lesson limits input to 10,000 characters. The reference rejects duplicate object keys rather than silently choosing one occurrence. Its intended setting is small educational save data, not an unrestricted hostile-input processing service. Resource limits and parser behavior need deeper review in a production security boundary.

For trusted course fixtures, the lesson's checks are sufficient to expose common mistakes: missing fields, wrong types, unsupported versions, invalid ranges and truncated files. Never use `eval` to “parse” a save. The intended input is data, not instructions to execute. The JSON format does not require evaluating arbitrary Python expressions.

Writing safely is a separate problem

The lesson functions only encode and decode strings. A file wrapper controls where bytes are read or written. The capstone reference demonstrates writing a validated payload to a temporary file in the destination directory and replacing the destination only after the temporary write succeeds. This protects the old save from ordinary failed-write paths tested here; it is not a universal power-loss durability guarantee or a concurrent-writer transaction system.

Keep those claims distinct. A round trip proves that one valid state can be encoded and decoded by your implementation. It does not prove malformed data is rejected, interrupted writes are safe, all future versions are supported or the state matches the original game rules.

Worked example

Predict before running `examples/example_03_3.py`.

```
import json

original = {"version": 1, "x": 2, "health": 90}
text = json.dumps(original, sort_keys=True)
candidate = json.loads(text)
print("encoded:", text)
print("same values:", candidate == original)
print("independent dictionary:", candidate is not original)

live = dict(original)
try:
    live = json.loads('{"version":')
except json.JSONDecodeError:
    print("Rejected truncated JSON; live health:", live["health"])
```

The example demonstrates representation and assignment timing, not complete schema validation. `json.loads` can return many kinds of JSON values, so the lab must add explicit schema checks after parsing. The malformed input raises before the assignment to `live`, leaving the earlier dictionary available to the caller.

Expected demonstration output

```
encoded: {"health": 90, "version": 1, "x": 2}
same values: True
independent dictionary: True
Rejected truncated JSON; live health: 90
```

Practical work and pass evidence

Implement `encode_save` and `decode_save` in `labs/student/lab_03_3.py`, including the exact schema described above, duplicate-key rejection and the input-length limit. Return new dictionaries and do not write files inside these functions. Use a separate disposable workspace file to exercise a file wrapper.

Test a valid round trip, missing and extra fields, wrong types, a Boolean version, unsupported version, out-of-range health, duplicate keys, excessive text and truncated input. Keep the known-good state and save file unchanged after a failed load. Pass only after explaining why successful JSON parsing is not the same as valid game data.

Changed-case challenge

Propose a version-2 save with a new field. Write the policy for rejecting or migrating version 1 before implementing any migration.

Companion source study

[S11] Python json library; [S08] Python: Errors and Exceptions; [S15] Python os library: replace. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. Does valid JSON imply a valid game state?
2. What does a round-trip test check?
3. Why include a format version?
4. Why is `{"version":1,"x":2,"health":-50}` invalid even though it is valid JSON?
5. What is the danger of resetting the live world to an empty dictionary before attempting a load?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.

Lesson 03.4: Classes, composition and state machines

Learning outcome: Represent behavior without creating an unnecessary inheritance hierarchy.

Objects group related state and behavior

A class defines how a kind of object is initialized and which operations it exposes. An instance is one particular object created from that definition. A controller object might own a movement state and expose a method for processing events. This grouping can make ownership clear, but a class is not automatically a better solution than a small function and dictionary.

Start with the behavior you need. Our movement controller has three states: idle, moving and stunned. It receives events such as move, stop, hit and recover. Its value is not an inheritance hierarchy; it is a precise transition policy that prevents contradictory combinations such as simultaneously being idle and moving.

Write the transition table first

The five allowed state changes are idle+move to moving, moving+stop to idle, idle+hit to stunned, moving+hit to stunned, and stunned+recover to idle. Other events in a valid state leave it unchanged. An unknown current state is an error. This difference is deliberate: an unhandled player request can be a valid no-op, while an impossible stored state suggests corrupt data or a programming defect.

A table makes gaps visible. Does stop do anything while stunned? In this policy it does not. Does recover while moving reset movement? It does not. You may choose another policy for a different game, but record it before writing scattered conditionals. A state machine is useful because it names what is allowed, not because its code has a particular shape.

A transition can be a pure function

`transition(state, event)` can return the next state without changing any object. A controller method can then assign that result to its own state. This separates rule testing from storage. Tests can enumerate the table without needing an entire player object, while the object can still provide a convenient application interface.

When invalid state is rejected, assignment should not install a partial result. Keep the same candidate-before-commit pattern used for loading. The principle is broader than files: validate an operation before replacing the authoritative state.

Composition separates independent concerns

A character can contain a movement controller and an inventory. Stunned movement does not necessarily imply that the inventory ceases to exist. Making inventory a component or owned object allows its rules to be tested without the movement controller. Inheriting from classes named `MovingCharacterWithInventory` and `StunnedCharacterWithInventory` would entangle independent choices.

Composition means assembling behavior from parts with defined interfaces. It does not mean putting every field into a different class. A simple game may use a character record containing a state value and a list. The important question is which part owns each fact and how another part is allowed to change it.

Keep instance state independent

If each controller should start idle, initialize that state for each instance. Be careful with shared mutable objects defined at class or function-default scope; one character's changes can become visible in another unexpectedly. Test two controllers and verify that moving one does not move the other. Later engine storage may deliberately share immutable data, but that is an explicit design, not accidental aliasing.

Test all five allowed transitions and at least three forbidden transitions. A test showing that idle can become moving does not establish that stunned movement is prohibited. Finish by tracing a longer event sequence and explaining each result from the same table, so the implementation is not merely matching isolated examples.

Worked example

Predict before running `examples/example_03_4.py`.

```
TRANSITIONS = {
    ("idle", "move"): "moving",
    ("moving", "stop"): "idle",
    ("idle", "hit"): "stunned",
    ("moving", "hit"): "stunned",
    ("stunned", "recover"): "idle",
}

class Controller:
    def __init__(self):
        self.state = "idle"

    def on_event(self, event):
        self.state = TRANSITIONS.get((self.state, event), self.state)

controller = Controller()
for event in ("move", "hit", "move", "recover"):
    controller.on_event(event)
    print(event, "->", controller.state)
```

`self` refers to the particular controller receiving the method call. Its state starts independently in `__init__`. The table lookup defaults to the current state for an unlisted event. The demonstration keeps state valid by construction; the lab function additionally rejects an unknown starting state at its interface.

Expected demonstration output

```
move -> moving
hit -> stunned
move -> stunned
recover -> idle
```

Practical work and pass evidence

Implement `transition` in `labs/student/lab_03_4.py` with the exact five-entry table and the stated no-op/error policy. Build a small controller wrapper in your workspace and add an inventory field by composition rather than a hierarchy of movement subclasses.

Test every allowed transition, at least three forbidden transitions and an unknown state. Create two controller instances and demonstrate independence. Trace move, hit, move, recover, move, stop from idle. Pass by explaining both the state table and which object or function owns each change.

Changed-case challenge

Add an independent inventory operation while the controller is stunned. Decide explicitly whether it is allowed; do not let inheritance structure make the game-design decision accidentally.

Companion source study

[S09] Python: Classes; [S07] Python: Data Structures. Read the sections relevant to the named concepts; record the section studied and one additional example.

Five-question lesson quiz

Answer independently. Core questions 1-3: at least 5/6. All five: at least 8/10, plus practical evidence.

1. What does encapsulation protect?
2. Why can independent booleans create contradictory states?
3. When is composition useful?
4. Under the assigned table, what happens to a stunned controller receiving move, followed by recover?
5. Why test two controller instances instead of only one?

Evidence to retain: predictions, code or worksheet, actual output, changed-case reasoning, quiz answers and corrections.